MosicaFE

统一调用 FEM、VEM 与 RT0 求解偏微分方程的 Python 库

使用手册与开发指南

MosicaFE contributors

2026 年 10 月 25 日

摘要. 本手册介绍 MosicaFE——一个把有限元（FEM）、虚拟元（VEM）与最低阶 Raviart–Thomas 混合元（RT0×P0）统一到同一套接口下的 Python 库。用户只需在主文件里写几行代码：选择网格、选择" 元"、选择线性求解器、选择可视化工具，即可离散并求解 Poisson 型偏微分方程。库的分层设计（`MosicaFE.core`、`MosicaFE.spaces`、`MosicaFE.physics`、`MosicaFE.solvers`、`MosicaFE.postprocess`）使得**新增一种单元**或**新增一门方程**都是局部改动。手册给出完整的调用示例、数学背景、精度与收敛性验证，以及扩展框架的模板。

目录

第一章 引言 **1**
1.1 概述 . . . 1
1.2 设计哲学 . . . 1
1.3 快速开始 . . . 2
1.4 包结构 . . . 3
1.5 本手册的组织 . . . 3

第二章 安装与环境 **5**
2.1 系统要求 . . . 5
2.1.1 Python 版本 . . . 5
2.1.2 依赖 . . . 5
2.2 安装 . . . 5
2.2.1 方式一：以包的形式安装（推荐） . . . 5
2.2.2 方式二：直接使用源码 . . . 5
2.3 验证安装 . . . 6
2.4 编译本手册 . . . 6

第三章 数学背景 **7**
3.1 模型问题 . . . 7
3.2 弱形式 . . . 7
3.3 有限元离散（FEM） . . . 7
3.4 任意阶虚拟元离散（VEM） . . . 8
3.4.1 局部空间与自由度 . . . 8
3.4.2 缩放单项式与梯度投影 . . . 9
3.4.3 L^2 投影、刚度、质量与载荷 . . . 9
3.5 RT0×P0 混合离散 . . . 10
3.6 误差估计 . . . 11

第四章 核心模块：网格与求积 **12**
4.1 网格数据结构 . . . 12
4.1.1 网格的组成部分 . . . 12

4.1.2 VEM 网格通用性与加密 . . . 12
4.1.3 facet 拓扑 . . . 13
4.1.4 面拓扑与定向：`core/facets.py` . . . 13
4.2 网格生成 . . . 14
4.3 网格属性与几何量 . . . 15
4.4 网格加密 . . . 16
4.5 参考单元 . . . 16
4.6 求积规则 . . . 17

第五章 离散空间 19
5.1 空间抽象 . . . 19
5.2 Lagrange 元（经典有限元） . . . 19
5.2.1 P1 三角形 . . . 19
5.2.2 P2 三角形 . . . 20
5.2.3 张量积元 . . . 20
5.2.4 自由度的全局编号 . . . 20
5.3 虚拟元空间 . . . 21
5.4 RT0×P0 混合空间 . . . 21
5.4.1 空间层给出的两个块 . . . 22
5.5 空间工厂 `FESpace` . . . 23

第六章 物理问题与组装 24
6.1 问题接口 . . . 24
6.2 Poisson 问题 . . . 24
6.3 边界条件 . . . 25
6.3.1 强加 Dirichlet 条件的正确做法 . . . 25
6.3.2 其它边界条件 . . . 25
6.4 组装算法 . . . 26
6.5 使用 Poisson 问题 . . . 26
6.6 PDE 注册表 . . . 27

第七章 线性求解器 28
7.1 求解器接口 . . . 28
7.2 直接法 . . . 28
7.3 迭代法 . . . 29
7.4 求解器选择指南 . . . 29

第八章 误差分析与收敛性 30
8.1 误差范数 . . . 30
8.2 收敛性研究框架 . . . 30

8.3 收敛阶估计 31
8.4 实测验证结果 31

第九章 可视化 **33**
9.1 可视化工具一览 33
9.2 画解 33
9.3 画收敛曲线 34

第十章 完整示例 **35**
10.1 示例一：两行求解 Poisson 方程 35
10.2 示例二：换一种元（同一段代码，三种方法） 35
10.3 示例三：任意阶 VEM 的二维与三维主程序 36
10.3.1 细网格结果与误差图 38
10.4 示例四：自定义新 PDE（Helmholtz） 41
10.5 示例五：反应扩散方程 41

第十一章 进阶话题 **43**
11.1 精度陷阱与经验教训 43
11.1.1 陷阱一：Dirichlet 消去漏掉右端项修正 43
11.1.2 陷阱二：张量积单元的求积阶不足 43
11.1.3 陷阱三：高阶投影的常数规范化 43
11.1.4 陷阱四：把高阶积分矩当作点值 43
11.1.5 陷阱五：numpy 的重复下标赋值 44
11.2 网格细化策略 44
11.3 性能与向量化 44
11.4 与其它库的关系 44

第十二章 扩展框架 **45**
12.1 新增一种单元 45
12.2 新增一门方程 46
12.3 新增一种混合格式（以 RT0×P0 为模板） 47
12.4 新增一种网格 47
12.5 开发流程与测试要求 48

第十三章 常见问题与排查 **49**
13.1 常见问题 49
13.1.1 导入错误 49
13.1.2 收敛阶不对 49
13.1.3 迭代法不收敛 49
13.1.4 内存不足 50
13.2 常见问答 50

附录 A API 速查 **51**
A.1 顶层模块 `MosicaFE` 51
A.2 `Mesh` 52
A.3 `FESpace` 与空间方法 52
A.4 物理问题 53
A.5 求解器与后处理 54

附录 B 数学符号 **55**
B.1 一般记号 55
B.2 函数空间 55
B.3 算子与范数 55

附录 C 参考文献 **57**

第一章　引言

1.1　概述

MosicaFE 是面向教学与科研的偏微分方程求解库。它把原本互相独立的三套程序（经典有限元、虚拟元、RT0 混合有限元）整合成**一个包、一套接口**，并使用同一套网格、求解器与后处理工具。

库的核心设计目标是：

(1) **调用极简**：主文件通常不超过 10 行；

(2) **模块解耦**：网格、单元、方程、线性代数、后处理各自独立；

(3) **易于扩展**：新增单元或新增 PDE 都只需实现少量接口；

(4) **可验证**：内置收敛性研究与误差范数，收敛阶可一键复现；

(5) **自包含**：只依赖 `numpy`（`scipy` 与 `matplotlib` 可选）。

1.2　设计哲学

MosicaFE 的设计对应有限元方法的数学结构：

$$\underbrace{\text{PDE}}_{\text{physics}} \xrightarrow{\text{弱形式}} \underbrace{\text{变分问题}}_{\text{spaces}} \xrightarrow{\text{离散化}} \underbrace{Au=b}_{\text{assembly}} \xrightarrow{\text{求解}} \underbrace{u_h}_{\text{postprocess}} \tag{1.1}$$

每一步由独立的模块负责，模块之间只通过明确的接口通信：

- `MosicaFE.core` 提供网格与求积规则；
- `MosicaFE.spaces` 把” 选哪种单元” 翻译成**矩阵级接口**：刚度矩阵、质量矩阵、载荷向量（详见第五章）；
- `MosicaFE.physics` 只做这些矩阵的线性组合，因此描述一门新 PDE 通常只要十几行；
- `MosicaFE.solvers` 负责线性代数；
- `MosicaFE.postprocess` 负责误差、收敛性与可视化。

要点

”元”与”方程”的正交分解是 MosicaFE 与三套原始程序最大的区别。原程序把”三角形网格 + 线性元 + Poisson + spsolve + 画图”写死在同一个文件里；现在这几件事互不相干：换一种元不必改方程，换一门方程不必改单元。

1.3 快速开始

下面是一个完整的 2D Poisson 求解程序：

```
import numpy as np
from MosicaFE import Mesh, solve_poisson

source = lambda x: 2*np.pi**2 * np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

u = solve_poisson(Mesh.rectangle(nx=32, ny=32), element="P2", f=source)
print("deg of freedom:", u.size)
```

Listing 1.1: 示例 1.1：两行求解 Poisson 方程

等价的”分步”写法（也是本手册后续使用的标准形式）：

```
import numpy as np
from MosicaFE import Mesh, FESpace, PoissonProblem
from MosicaFE.postprocess import plot_solution

def exact(x):
    return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

def source(x):
    return 2*np.pi**2 * exact(x)

mesh = Mesh.rectangle(nx=32, ny=32)          # (1) 选网格
V = FESpace.lagrange(mesh, degree=2)         # (2) 选元
problem = PoissonProblem(V, f=source, g=lambda x: 0.0)
u = problem.solve(method="direct")           # (3) 选求解器
plot_solution(u, mesh, V, filename="p2.png")  # (4) 选可视化

print(problem.compute_errors(u, exact))
```

Listing 1.2: 示例 1.2：选择网格 / 元 / 求解器 / 可视化

把第 1.2 行中的 `FESpace.lagrange(mesh,degree=2)` 换成 `FESpace.vem(mesh)`，得到的就不再是有限元，而是**同一个方程**在**同一种网格**上的虚拟元解；程序其它部分一字不改。

要点

这就是本库存在的意义：让”换方法”的代价降到一行代码。

1.4 包结构

表 1.1: MosicaFE 的模块划分

模块	职责
`MosicaFE.core`	网格数据结构与生成（区间 / 矩形 / 长方体 / 多边形 / Voronoi）、**面拓扑与定向**（`facets.py`，$H(\mathrm{div})$ 协调空间的基础）、求积规则、几何工具（面积、体积、雅可比、梯度变换）
`MosicaFE.spaces`	离散空间：`LagrangeSpace`（P1/P2/Q1/Q2）、`VEMSpace`（任意正整数 k 阶增强型虚拟元）、`RT0Space`（RT0×P0），以及工厂 `FESpace`
`MosicaFE.physics`	PDE 弱形式与组装：`PoissonProblem`、`MixedPoissonProblem`、`ReactionDiffusionProblem`，含 PDE 注册表
`MosicaFE.solvers`	线性求解器：直接法、CG、GMRES、BiCGSTAB、MINRES 与自动选择
`MosicaFE.postprocess`	误差范数（L^2 / H^1 / L^∞）、收敛性研究、可视化、混合格式（压力 + 通量）的误差范数

要点

FEM、VEM、RT0 三类元**全部是库自身的实现**：RT0×P0 的显式基函数、精确质量矩阵、面拓扑与鞍点组装分别位于 `spaces/rt0.py`、`core/facets.py` 与 `physics/mixed_poisson.py`，MosicaFE 不依赖、也不需要接入任何外部有限元后端。

1.5 本手册的组织

- 第二章：安装与环境；
- 第三章：三种方法的数学背景；
- 第四–五章：网格、求积与离散空间；
- 第六–七章：方程组装与线性求解；
- 第八–九章：误差分析与可视化；
- 第十章：完整示例；

- 第十一–十二章：进阶话题与框架扩展；
- 第十三章：常见问题；
- 附录 A–C：API 速查、数学符号、参考文献。

第二章　安装与环境

2.1　系统要求

2.1.1　Python 版本

要求 Python $\geq$ 3.9。开发与验证使用的环境为 Python 3.12.10（conda 环境 `fealpy`），`numpy` 2.2.4、`scipy` 1.15.2、`matplotlib` 3.10.1。

2.1.2　依赖

表 2.1: 依赖关系

包	是否必需	用途
`numpy`	必需	数组、稠密线性代数
`scipy`	可选	稀疏矩阵、`spsolve`、Krylov 求解器、Voronoi 网格
`matplotlib`	可选	可视化

缺少 `scipy` 时，MosicaFE 自动退化为纯 `numpy` 的稠密实现（`MosicaFE._compat` 统一处理），小规模问题照常可解；缺少 `matplotlib` 时，所有绘图函数打印一条提示并返回 `None`，不会报错。

2.2　安装

2.2.1　方式一：以包的形式安装（推荐）

```
cd D:\mosica\MosicaFE
python -m pip install -e .
```

2.2.2　方式二：直接使用源码

把 MosicaFE 的**父目录**加入模块搜索路径即可，无需安装：

```
# PowerShell
cd D:\mosica
```

```
$env:PYTHONPATH = "D:\mosica"
& "D:\anaconda\envs\fealpy\python.exe" MosicaFE\examples\poisson_demo.py
```

2.3 验证安装

MosicaFE 自带两个不依赖第三方测试框架的脚本：

```
& "D:\anaconda\envs\fealpy\python.exe" MosicaFE\tests\run_tests.py
& "D:\anaconda\envs\fealpy\python.exe" MosicaFE\tests\run_checks.py
```

前者跑 54 个单元/集成测试，后者打印各方法的实测收敛阶。预期输出片段：

```
=== P2 ===
  level        h                 L2    rate          H1    rate        Linf    rate
      0  0.35355    3.4643e-02       -  4.4391e-01       -  1.4645e-01       -
      1  0.17678    4.4076e-03    2.97  1.1632e-01    1.93  3.8060e-02    1.94
      2  0.08839    5.5040e-04    3.00  2.9416e-02    1.98  9.6074e-03    1.99
      3  0.04419    6.8699e-05    3.00  7.3752e-03    2.00  2.4076e-03    2.00
  rate[L2] (last 3) = 3.002
  rate[H1] (last 3) = 1.990
```

如果安装了 `pytest`，也可以直接：

```
python -m pytest MosicaFE/tests -q
```

2.4 编译本手册

本文档使用 `ctexbook` 文档类，正文含中文，需要用 XeLaTeX 编译两遍：

```
xelatex MosicaFE_Guidebook_zh.tex
xelatex MosicaFE_Guidebook_zh.tex
```

在 TeXShop 中把引擎选为 `XeLaTeX` 并点击 Typeset 即可。

第三章　数学背景

3.1　模型问题

本库（以及原来的三套程序）求解的核心模型问题是 Poisson 方程

$$\begin{cases} -\nabla\cdot(\kappa\nabla u) = f, & \text{in } \Omega, \\ u = g, & \text{on } \partial\Omega, \end{cases} \tag{3.1}$$

其中 $\Omega\subset\mathbb{R}^d$（$d=1,2,3$）是有界区域，$\kappa>0$ 是扩散系数。当 $g=0$ 且 $\kappa=1$ 时即经典的 $-\Delta u=f$。

3.2　弱形式

定义 3.1 (弱形式)**.** 记 $V=\{v\in H^1(\Omega): v|_{\partial\Omega}=0\}$，$V_g=\{v\in H^1(\Omega): v|_{\partial\Omega}=g\}$。求 $u\in V_g$ 使得

$$a(u,v)=\ell(v),\qquad \forall v\in V, \tag{3.2}$$

其中

$$a(u,v)=\int_\Omega \kappa\nabla u\cdot\nabla v\,\mathrm{d}x,\qquad \ell(v)=\int_\Omega fv\,\mathrm{d}x. \tag{3.3}$$

ℓ 也可以带 Neumann 数据项 $\int_{\Gamma_N} hv\,\mathrm{d}s$，MosicaFE 的物理层接口为此预留了扩展点。

3.3　有限元离散（FEM）

设 $V_h\subset V$ 是有限维子空间，基函数为 $\{\varphi_i\}_{i=1}^n$。离散问题为：求 $u_h=\sum_j u_j\varphi_j$ 使得

$$\sum_{j=1}^{n} u_j\, a(\varphi_j,\varphi_i)=\ell(\varphi_i),\qquad i=1,\dots,n, \tag{3.4}$$

即线性方程组

$$Au=b,\qquad A_{ij}=\int_\Omega \kappa\nabla\varphi_j\cdot\nabla\varphi_i\,\mathrm{d}x,\qquad b_i=\int_\Omega f\varphi_i\,\mathrm{d}x. \tag{3.5}$$

在参考单元 $\hat K$ 上计算时用到梯度变换

$$\nabla_x\varphi_i = J_K^{-\top}\nabla_{\hat x}\hat\varphi_i,\qquad \int_K g\,\mathrm{d}x=\int_{\hat K} g\circ F_K\,|\det J_K|\,\mathrm{d}\hat x. \tag{3.6}$$

表 3.1: MosicaFE 中 Lagrange 元的信息

单元	名字	每个单元的局部自由度	自由度类型
三角形	P1	3	顶点
三角形	P2	6	顶点 + 棱中点
四边形	Q1	4	顶点
四边形	Q2	9	顶点 + 棱中点 + 单元中心
四面体	P1	4	顶点
四面体	P2	10	顶点 + 棱中点
六面体	Q1	8	顶点
六面体	Q2	27	顶点 + 棱 + 面 + 体

3.4 任意阶虚拟元离散（VEM）

本库实现二维、三维的协调增强型虚拟元。设置正整数 `degree=k` 即选择阶数，不再限于一阶。整体流程仍为网格、空间、方程、求解和后处理；局部计算由 `spaces/vem_local.py` 的 `LocalVEM` 递归完成，`spaces/vem.py` 的 `VEMSpace` 管理共享自由度与全局组装。虚拟基函数无需显式求值，所需积分通过多项式投影和自由度获得。

3.4.1 局部空间与自由度

设 $\mathcal{P}_r(E)$ 为实体 E 上总次数不超过 r 的多项式，$r<0$ 时为空间 $\{0\}$。取 $\mathcal{B}_k^*(K)$ 为缩放单项式基中总次数恰为 $k-1$ 或 k 的部分。二维单元、三维单元及三维的二维面递归使用增强空间

$$\begin{aligned} V_k(K)=\{v\in H^1(K): v|_{\partial K}\ \text{continuous},\quad v|_F\in V_k(F),\quad \Delta v\in\mathcal{P}_k(K),\\ (v-\Pi_k^{\nabla}v,m)_K=0\quad \forall m\in\mathcal{B}_k^*(K)\}. \end{aligned} \tag{3.7}$$

二维中 F 是边，$V_k(F)=\mathcal{P}_k(F)$；三维中 F 是平面多边形面。增强约束使高阶矩可由梯度投影补齐，不增加未知量。自由度依次为：

1. 每个顶点的函数值；每条边内部 $k-1$ 个等距点的函数值。
2. 二维单元、三维面上的归一化矩 $|E|^{-1}\int_E vm_\alpha$，$|\alpha|\le k-2$。
3. 三维单元内部同样的归一化体矩，$|\alpha|\le k-2$。

共享顶点、边、面的自由度只分配一次。$k=1$ 时只有顶点自由度。若单元有 n_v 个顶点、n_e 条边、n_f 个面，则

$$N_K^{2D} = n_v + n_e(k-1) + \frac{k(k-1)}{2},$$
$$N_K^{3D} = n_v + n_e(k-1) + n_f\frac{k(k-1)}{2} + \frac{(k-1)k(k+1)}{6}.$$

3.4.2 缩放单项式与梯度投影

`powers(dim,k)` 枚举 $|\alpha| \le k$，因此算法不需要为每个阶数另写一套公式。实体中心 c_E 取顶点坐标的算术平均，h_E 取直径，$m_\alpha(x) = \prod_j (((x-c_E)Q_E)_j/h_E)^{\alpha_j}$。三维面上的 Q_E 是由 SVD 得到的正交平面基；体单元使用坐标轴。梯度投影满足

$$(\nabla\Pi_k^\nabla v, \nabla q)_K = -(v, \Delta q)_K + \sum_{F\subset\partial K}(v, \partial_n q)_F, \qquad q \in \mathcal{P}_k(K). \tag{3.8}$$

Δq 次数不超过 $k-2$，由内部矩计算；边界项由子实体的投影计算。边的 $k+1$ 个点值唯一确定 k 次多项式迹：

$$v|_e = \sum_{i=0}^{k} v(x_i)\ell_i, \qquad \int_e v\,\partial_n q = \sum_{i=0}^{k} v(x_i)\int_e \ell_i\,\partial_n q. \tag{3.9}$$

$$\begin{cases} \displaystyle\int_K \Pi_k^\nabla v = \int_K v, & k \ge 2, \\ \displaystyle\frac{1}{n_v}\sum_i (\Pi_1^\nabla v)(X_i) = \frac{1}{n_v}\sum_i v(X_i), & k = 1. \end{cases} \tag{3.10}$$

最后一个条件固定梯度无法决定的常数。记 $D_{i\alpha} = \mathrm{dof}_i(m_\alpha)$、$H_{\alpha\beta} = (m_\alpha, m_\beta)_K$、$G_{\alpha\beta} = (\nabla m_\alpha, \nabla m_\beta)_K$。用均值条件替换 G 的常数行，得到 $\widetilde{G}$ 及对应右端 B：

$$P_\nabla = \mathrm{solve}(\widetilde{G}, B), \qquad \Pi_k^\nabla v = \sum_\alpha (P_\nabla \mathbf{v})_\alpha m_\alpha. \tag{3.11}$$

这是线性方程求解，不是旧版伪逆或线性最小二乘重构。应检查 $P_\nabla D = I$，即再生整个 $\mathcal{P}_k$。

3.4.3 L^2 投影、刚度、质量与载荷

先取 $C = HP_\nabla$，再把 $|\alpha| \le k-2$ 的行替换为真实矩对应的 $|K|$ 倍自由度选择行；高阶行保留增强约束给出的值。于是 $P_0 = \mathrm{solve}(H, C)$。

$$A_K = P_\nabla^T G P_\nabla + S_K, \tag{3.12}$$

$$R_\nabla = I - DP_\nabla, \qquad S_K = \sigma R_\nabla^T R_\nabla, \tag{3.13}$$

$$\sigma = \frac{\mathrm{tr}(P_\nabla^T G P_\nabla)}{\max(\mathrm{tr}(R_\nabla^T R_\nabla), 10^{-30})}. \tag{3.14}$$

$$\begin{aligned} M_K &= P_0^T H P_0 + |K|(I-DP_0)^T(I-DP_0), \\ b_K &= P_0^T \mathbf{f}_K, \qquad (\mathbf{f}_K)_\alpha = \int_K f m_\alpha. \end{aligned} \tag{3.15}$$

默认 `trace` 稳定化对称半正定；`legacy` 现在是它的兼容别名。当投影残差范数小于 10^{-10} 时不加稳定化。`none` 只返回一致性项，一般不能控制投影核，仅用于诊断。代码核心对应如下：

```
energy = np.linalg.solve(system, rhs)
moments = H @ energy
for j in range(len(self.low)):
    moments[j] = 0.0
    moments[j, self.moment_start+j] = self.measure
l2 = np.linalg.solve(H, moments)
consistent = energy.T @ G @ energy
residual = np.eye(len(self.dofs)) - D @ energy
stable = residual.T @ residual
scale = np.trace(consistent) / max(np.trace(stable), 1e-30)
```

Listing 3.1: 局部投影与稳定化的核心计算

默认求积参数为 $2k+2$，高阶时采用动态 Gauss/Duffy 求积。多边形分三角形、多面体分四面体仅用于积分，不引入有限元子网格未知量。`VEMSpace` 沿用 `BaseFESpace` 组装；Dirichlet 条件同时设置边界点值及边界面的归一化矩，不能把矩自由度当作某个坐标处的函数值。

3.5 RT0×P0 混合离散

混合形式把 Poisson 方程写成一阶系统

$$q = -\nabla p, \qquad \nabla \cdot q = f \ \text{ in } \Omega, \qquad p = 0 \ \text{ on } \partial\Omega. \tag{3.16}$$

弱形式：求 $(q,p) \in RT0 \times P0$ 使得

$$\int_\Omega \nu \cdot q \,\mathrm{d}x - \int_\Omega p \nabla \cdot \nu \,\mathrm{d}x = 0, \qquad \int_\Omega r \nabla \cdot q \,\mathrm{d}x = \int_\Omega f r \,\mathrm{d}x \tag{3.17}$$

对任意 (ν, r) 成立。离散后得到鞍点系统

$$\begin{pmatrix} M & B^\top \\ B & 0 \end{pmatrix} \begin{pmatrix} \mathbf{Q} \\ \mathbf{P} \end{pmatrix} = \begin{pmatrix} 0 \\ \mathbf{f} \end{pmatrix}, \tag{3.18}$$

它是**对称不定**的，因此不能使用共轭梯度法。

表 3.2: RT0×P0 的自由度和边界条件

量	自由度位置	个数
通量 q_h	每个单元面（2D 边 / 3D 三角面），相邻单元共享	#facets
压力 p_h	每个单元一个常数	#cells
Dirichlet 条件 $p=0$	混合格式的**自然**边界条件，无需消去	

3.6 误差估计

定理 3.1 (Cea 引理). 设 u 是精确解、u_h 是有限元解，则 $\|u-u_h\|_V \le C\inf_{v_h\in V_h}\|u-v_h\|_V$。

定理 3.2 (先验误差估计). 若 $u\in H^{k+1}(\Omega)$、采用 k 次多项式，则

$$\|u-u_h\|_{L^2}\le Ch^{k+1}|u|_{H^{k+1}},\qquad |u-u_h|_{H^1}\le Ch^{k}|u|_{H^{k+1}}. \tag{3.19}$$

例 3.1 (本库实测的收敛阶).

- P1（三角形）：L^2 为 $O(h^2)$，H^1 为 $O(h)$；
- P2 / Q2：L^2 为 $O(h^3)$，H^1 为 $O(h^2)$；
- VEM（任意 $k\ge 1$，合格多边形/多面体）：在正则性及稳定性条件下，L^2 为 $O(h^{k+1})$，H^1 为 $O(h^k)$；
- RT0×P0：压力 L^2 为 $O(h)$，通量 L^2 为 $O(h)$。

第八 章给出实测数据。

第四章　核心模块：网格与求积

4.1　网格数据结构

4.1.1　网格的组成部分

Mesh 对象保存以下信息：

表 4.1： Mesh 的主要属性

属性	类型	含义
nodes	(n_nodes, dim) 数组	顶点坐标
elements	list[ndarray]	每个单元的顶点编号，**变长**
element_type	str	interval/triangle/quad/tet/hex/polygon/polyhedron
boundary_nodes	ndarray	边界顶点编号（由 facet 拓扑自动判定）
dim	int	空间维数

要点

elements 故意设计成**变长列表**：三角形、四边形、五边形、Voronoi多边形、四面体、六面体因此可以共用同一套组装循环。如果所有单元顶点数相同，可以通过 mesh.uniform_vertices 取出 (n_cells,k) 的数组视图。

4.1.2　VEM 网格通用性与加密

elements 的变长连接表允许不同顶点数的单元共存。三维面表有公共模板和逐单元面表两种格式（见下方代码），其中顶点号均为单元内的局部编号。显式面表优先于按顶点数推断；例如 8 个顶点也可能组成七棱锥，不能直接当六面体。mesh.get_cell_faces(c) 返回该单元的真实面表。

```
mesh2 = Mesh.voronoi_polygon(n_points=1024, seed=42)
mesh3 = Mesh.voronoi_polyhedron(n_points=192, seed=42)
```

```
# Common template: faces[face][vertex].
# Per-cell faces: faces[cell][face][vertex].
# mesh3 = Mesh(nodes, elements, element_type="polyhedron", faces=faces)
```

Listing 4.1: 二维与三维多边形网格入口

当前几何条件：二维单元和三维面为直边、平面多边形；各自的顶点均值必须严格位于星形核内。三维体单元也须满足此星形条件，表面必须闭合、连通且可定向，每条表面边恰接两个面。允许满足条件的凹单元。相邻单元共享面需具有相同的全局顶点和边界环，一个面最多属于两个单元。程序按边邻接统一局部面方向，以有符号体积确定外向。不支持带洞实体、曲边、非平面面，也不会自动修复裂缝、重叠或悬挂节点。

二维 Voronoi 用 8 个方向的镜像点封闭边界，三维用 26 组镜像点生成多面体，三维生成器由 `core/polyhedral.py` 实现，依赖 SciPy。体积与积分按真实面表的扇形四面体计算。

结构化网格使用 `refine_uniform()` 将各方向区间数加倍。Voronoi 加密则提高 `n_points` 并重新生成网格；同一个随机种子也不构成嵌套细化，没有父子单元关系或解转移。当前没有局部自适应加密。`get_mesh_size()` 返回最大单元直径，不能把 $N^{-1/d}$ 当作其精确值。

4.1.3 facet 拓扑

余维 1 的单元面（facet）是边界判定的基础：1D 的 facet 是端点、2D 是边、3D 是平面多边形面。`mesh.facets` 给出全部 facet，`mesh.boundary_facets` 给出只被一个单元拥有的 facet，它们的顶点集合就是 Dirichlet 边界所在的位置。

性质 4.1. 对二维任意多边形网格，facet 就是多边形的边；对六面体网格按固定的 6 个面表展开；对一般三维多面体，可以通过 `Mesh(...,faces=...)` 显式提供面表。

> **注意**
>
> 上面这套 `mesh.facets` 是”**边界判定**用的顶点集合”，高阶 VEM 还需共享边点与面矩，边界条件由 `VEMSpace.boundary_dofs` 和 `boundary_values` 统一处理。但对 $H(\mathrm{div})$ 协调的混合元（RT0）来说，还额外需要”**哪些单元共享同一个面**”以及”这个面的全局法向指向谁”，因此本库另有一层面拓扑：`core/facets.py`。下一小节给出它的用法。

4.1.4 面拓扑与定向：`core/facets.py`

RT0 的自由度是**面法向通量** $\Phi_f = \int_f q \cdot n_f \, \mathrm{d}s$，相邻两个单元必须共享同一个未知量。这要求显式地给出面的全局编号与定向。本库把它做成 `core` 层的一等公民 `FacetTopology`（`mesh.facet_topology` 惰性构建并缓存）：

```
from MosicaFE import Mesh

mesh = Mesh.box(nx=2, ny=2, nz=2, element_type="tet")
t = mesh.facet_topology

```

表 4.2: FacetTopology 的主要属性（t = mesh.facet_topology）

属性 / 方法	含义
t.facets	(n_facets,d)：每个面的**升序**顶点表（全局唯一键）
t.cell_facets	(n_cells,d+1)：[c,i] 是单元 c 第 i 个局部面的全局编号
t.signs	±1：全局法向是否指向该单元外部
t.normals	未归一化的全局法向（模长 = 面测度）
t.counts	相邻单元个数：内部面 2、边界面 1
t.boundary_facets()	边界面编号数组（t.n_boundary 为个数）

```
print(t.summary())          # cells=48, facets=120, interior=72, boundary=48
print(t.cell_facets[0])     # 单元 0 的 4 个局部面的全局编号
print(t.signs[0])           # 外法向符号（+1 / -1）

# 内部面的两侧符号一定相反 —— 这就是 H(div) 协调性的离散体现
c0, c1 = t.facet_cells[0]
print(t.signs[c0][list(t.cell_facets[c0]).index(0)] *
      t.signs[c1][list(t.cell_facets[c1]).index(0)])   # -1
```

Listing 4.2: 示例 4.2：面拓扑与定向

要点

build_facet_topology 会拒绝两类”会静默算错”的输入：非单纯形网格（四边形 / 多边形 / 六面体），以及存在被 3 个以上单元共用的面（不协调剖分）。这是 RT0 只接受三角形 / 四面体网格的技术原因。

4.2 网格生成

所有网格都由 Mesh 的类方法生成，且统一采用**逆时针（2D）/ 正体积（3D）**取向。

```
from MosicaFE import Mesh

m1 = Mesh.interval(a=0.0, b=1.0, nx=20)                # 1D interval

m2 = Mesh.rectangle(nx=8, ny=8)                        # triangles
m3 = Mesh.rectangle(nx=8, ny=8, element_type="quad")   # quadrilaterals

m4 = Mesh.box(nx=4, ny=4, nz=4)                        # tets (Kuhn split)
m5 = Mesh.box(nx=4, ny=4, nz=4, element_type="hex")    # hexahedra
m6 = Mesh.box(nx=3, ny=3, nz=3, pattern="center")      # 12 tets per cube

```

```
m7 = Mesh.pentagon(nx=8, ny=8)                          # pentagon mesh
m8 = Mesh.voronoi_polygon(n_points=100, seed=42)        # Voronoi polygons

for m in (m1, m2, m3, m4, m5, m7, m8):
    print(m.summary())
```

Listing 4.3: 示例 4.1：生成各类网格

表 4.3: 网格工厂速查

工厂	区域	单元
`Mesh.interval(a,b,nx)`	$[a,b]$	线段
`Mesh.rectangle(...,"triangle")`	矩形	三角形
`Mesh.rectangle(...,"quad")`	矩形	四边形
`Mesh.box(...,"tet")`	长方体	四面体
`Mesh.box(...,"hex")`	长方体	六面体
`Mesh.pentagon(nx,ny)`	单位正方形	五边形
`Mesh.voronoi_polygon(...)`	单位正方形	Voronoi多边形
`Mesh.voronoi_polyhedron(...)`	单位立方体	Voronoi 多面体
`Mesh.from_arrays(...)`	自定义	自动推断

注 4.1 (Kuhn 剖分)**.** `Mesh.box(...,element_type="tet")` 默认把每个小立方体剖成 6 个**共享主对角线**的四面体（Kuhn 剖分）。这样相邻立方体在公共面上的三角剖分方向一致，网格是协调的。`pattern="center"` 则额外插入立方体中心点，剖成 12 个四面体——与原始 VEM 三维脚本中的网格一致。

注 4.2 (Voronoi 网格)**.** Voronoi 网格生成时在区域外做 8 方向镜像反射点，保证边界处的 Voronoi 单元闭合，然后裁剪回区域。裁剪与排序之后会自动去掉相邻重合顶点，避免出现退化单元（一个含重复顶点的多边形会让扇形剖分出现零面积三角形）。

4.3 网格属性与几何量

```
mesh = Mesh.rectangle(nx=8, ny=8)

print(mesh.dim, mesh.n_nodes, mesh.n_cells)     # 2 81 128
print(mesh.get_mesh_size())                     # max cell diameter
```

```
print(mesh.cell_measures())                    # array of areas

v = mesh.get_cell_vertices(0)                  # (k, dim) coordinates
h = mesh.get_cell_diameter(0)
c = mesh.get_cell_centroid(0)
print(mesh.boundary_nodes[:5])
```

Listing 4.4: 示例 4.2：读取网格信息

4.4 网格加密

```
coarse = Mesh.rectangle(nx=4, ny=4)
fine = coarse.refine_uniform()
print(coarse.n_cells, fine.n_cells)     # 32 128
```

Listing 4.5: 示例 4.3：均匀加密

`refine_uniform()` 只对结构化网格（`interval` / `rectangle` / `box` / `pentagon`）有效：它把各方向的网格数翻倍后重新生成，因此三角形网格的单元数变为 4 倍、六面体网格变为 8 倍。对 Voronoi 这类非结构网格会抛出 `ValueError`。

4.5 参考单元

表 4.4: 参考单元约定

单元类型	参考单元
`interval`	$[0,1]$
`triangle`	$\{\xi \ge 0, \eta \ge 0, \xi+\eta \le 1\}$
`quad`	$[0,1]^2$，顶点顺序 $(0,0),(1,0),(1,1),(0,1)$
`tet`	标准四面体，顶点 $(0,0,0),(1,0,0),(0,1,0),(0,0,1)$
`hex`	$[0,1]^3$，先 $z=0$ 面逆时针 4 点，再 $z=1$ 面对应 4 点

单纯形使用仿射映射 $x = v_0 + J_K \hat{x}$；四边形与六面体使用 $[0,1]^d$ 上的双线性 / 三线性映射，其雅可比矩阵依赖参考点 $\hat{x}$，因此需要逐求积点计算。

```
import numpy as np
from MosicaFE.core.utils import (
    compute_jacobian, transform_gradient, apply_affine_map,
    polygon_area, polyhedron_volume,
)

```

```
quad = np.array([[0,0],[2,0],[2,2],[0,2]], float)
J, detJ = compute_jacobian(quad, np.array([0.5, 0.5]))
print(J, detJ)                      # [[2,0],[0,2]] 4.0
print(polygon_area(quad))           # 4.0
print(apply_affine_map([[0.5,0.5]], quad))   # [[1, 1]]
```

Listing 4.6: 示例 4.4：几何工具

注意

雅可比的约定。`compute_jacobian` 对两类单元返回**同一个约定**的雅可比：列是物理坐标对参考坐标的偏导，

$$J_{ab} = \frac{\partial x_a}{\partial \xi_b}, \qquad J = \left[\, \partial x/\partial \xi_1 \;\; \cdots \;\; \partial x/\partial \xi_d \,\right], \tag{4.1}$$

只有这样，梯度变换 `transform_gradient(dN,J)`（内部用 $J^{-\top}$）与四边形 / 六面体的 Newton 逆映射（内部用 J^{-1}）才是对的。**自己扩展代码时不要写成** $dN^{\top}V$——那是转置雅可比。对轴对齐的矩形 / 长方体，J 是对角阵、转置不变，这个错误会被完全掩盖；而网格一旦**旋转或畸变**（$J \neq J^{\top}$），梯度、刚度矩阵与插值就会静默算错。本库在 2026-09-21 修掉过这一处（第 7 号 bug），回归测试见 `tests/test_tensor_geometry.py`：它用有限差分核对 J，并检验旋转 $0°/15°/30°/45°$ 时误差逐位相同。

4.6 求积规则

```
from MosicaFE.core.quadrature import QuadratureRule, cell_quadrature

quad = QuadratureRule.get_quadrature("triangle", order=3)
print(quad.n_points, quad.weights.sum())     # 4 0.5

f = lambda xi: xi[0]**2 + xi[1]**2
value = sum(quad.weights[q] * f(quad.points[q]) for q in range(quad.n_points))
print(value)                                  # 1/6

# 映射到物理单元（自动处理多边形 / 多面体的扇形剖分）
mesh = Mesh.voronoi_polygon(n_points=20, seed=1)
pts, w = cell_quadrature(mesh, 0, order=3)
print(w.sum(), mesh.get_cell_measure(0))
```

Listing 4.7: 示例 4.5：使用求积规则

表 4.5: 可用的求积规则（`order` 为需精确积分的多项式次数）

单元	规则	说明
`interval`	Gauss–Legendre	$n = \lceil (order+1)/2 \rceil$ 点，$\int_0^1$ 归一
`triangle`	重心 / 3 点 / 4 点 / 6 点	分别精确到 1/2/3/4 次
`quad`	张量 Gauss	每个方向 n 点
`tet`	1 点 / 4 点 / 5 点	精确到 1/2/3 次
`hex`	张量 Gauss	每个方向 n 点

注意

张量积单元的求积阶需要按**每个方向**的次数来理解。`Q1` 单元刚度矩阵的被积函数在每个方向上是 2 次，所以每个方向至少需要 2 个 Gauss 点——MosicaFE 内部会自动把阶数取成 `max(2,2*p)`，用户一般不必手工指定。早期版本在这里少取了一个点，导致三维六面体的收敛阶算出来是负数。

第五章　离散空间

5.1　空间抽象

所有空间都继承 `BaseFESpace`，并向上提供**矩阵级接口**：

表 5.1: `BaseFESpace` 的关键方法

方法	含义
`stiffness_matrix()`	$A_{ij} = \int \nabla\varphi_j \cdot \nabla\varphi_i$
`mass_matrix()`	$M_{ij} = \int \varphi_j \varphi_i$
`load_vector(f)`	$b_i = \int f\varphi_i$
`boundary_dofs()`	Dirichlet 自由度编号
`interpolate(f)`	把函数插值到自由度上
`evaluate(cell,u,pts)`	在单元内把自由度重构成函数并求值
`evaluate_gradient(cell,u,pts)`	同上，返回梯度
`nodal_values(u)`	把自由度向量还原到网格节点（可视化用）
`get_cell_dofs(i)`	第 i 个单元的全局自由度编号
`get_dof_coordinates(k)`	第 k 个自由度的物理坐标
`eval_basis(xi)` / `eval_basis_grad(xi)`	参考单元上的基函数（仅显式基函数空间）

要点

这组接口是” 新增 PDE 只需十几行” 的根本原因：物理层永远只做矩阵的线性组合与边界处理，不必关心单元循环。

5.2　Lagrange 元（经典有限元）

5.2.1　P1 三角形

参考三角形上，用重心坐标 $\lambda_0 = 1 - \xi - \eta$，$\lambda_1 = \xi$，$\lambda_2 = \eta$：

$$\hat{\varphi}_0 = 1 - \xi - \eta, \qquad \hat{\varphi}_1 = \xi, \qquad \hat{\varphi}_2 = \eta. \tag{5.1}$$

5.2.2 P2 三角形

$$\begin{aligned} &\hat{\varphi}_0 = \lambda_0(2\lambda_0 - 1), && \hat{\varphi}_1 = \lambda_1(2\lambda_1 - 1), && \hat{\varphi}_2 = \lambda_2(2\lambda_2 - 1), \\ &\hat{\varphi}_3 = 4\lambda_0\lambda_1 \text{ (棱0-1)}, && \hat{\varphi}_4 = 4\lambda_1\lambda_2 \text{ (棱1-2)}, && \hat{\varphi}_5 = 4\lambda_2\lambda_0 \text{ (棱2-0)}. \end{aligned} \tag{5.2}$$

5.2.3 张量积元

四边形与六面体使用 1D Lagrange 基的张量积。以 Q2 为例，1D 节点取 $\{0, \frac{1}{2}, 1\}$：

$$N_0(t) = 2t^2 - 3t + 1, \qquad N_1(t) = -4t^2 + 4t, \qquad N_2(t) = 2t^2 - t, \tag{5.3}$$

单元基函数为 $N_{ijk}(\xi, \eta, \zeta) = N_i(\xi)N_j(\eta)N_k(\zeta)$。由于节点集是完整张量网格，$\{N_{ijk}\}$ 自动构成对偶于节点值的**节点基**。

5.2.4 自由度的全局编号

MosicaFE 用一个” 登记簿” 把几何实体映射到全局自由度：

- 顶点自由度：以全局顶点编号为 key；
- 棱自由度：以两个端点的**有序对**为 key；
- 面自由度（3D）：以面的 4 个顶点有序元组为 key；
- 单元内部自由度：以单元编号为 key。

于是相邻单元自动共享棱 / 面自由度，全局空间是 H^1 协调的。

```python
from MosicaFE import Mesh, FESpace

mesh = Mesh.rectangle(nx=8, ny=8)

V1 = FESpace.lagrange(mesh, degree=1)
V2 = FESpace.lagrange(mesh, degree=2)
print(V1.n_dofs, V2.n_dofs)      # 81 289

print(V2.get_cell_dofs(0))       # local -> global DOF map
print(V2.get_dof_coordinates(0))
print(V2.boundary_dofs()[:6])
```

Listing 5.1: 示例 5.1：创建空间并查看自由度

5.3 虚拟元空间

```python
import numpy as np
from MosicaFE import Mesh, FESpace

mesh = Mesh.voronoi_polygon(n_points=100, seed=42)
k = 3
V = FESpace.vem(mesh, degree=k)
# Equivalent: FESpace.create(mesh, family="vem", degree=k)
print(V.n_dofs, mesh.n_nodes)
A = V.stiffness_matrix()
M = V.mass_matrix()
b = V.load_vector(lambda x: 1.0)
one = V.interpolate(lambda x: 1.0)
print(np.max(np.abs(M @ one - b)))
```

Listing 5.2: 任意阶 VEM 空间及常函数检查

`degree` 必须为正整数，默认是 1；也可写 `element="VEM3"`。$k \geq 2$ 时，`u` 包含积分矩，因此通常 `V.n_dofs > mesh.n_nodes`。`V.interpolate` 计算函数的全部自由度；常函数的矩自由度不全为 1，不能用质量矩阵行和检查高阶一致性，应检查上面的 $M\mathbf{1}_{\mathrm{dof}} = b$。`V.nodal_values(u)` 返回实际顶点值，适合报告节点解的范围。

性质 5.1. 在合格网格及足够求积精度下，默认局部刚度对称半正定，常函数在核中，并对整个 $\mathcal{P}_k$ 满足多项式一致性。施加适当 Dirichlet 条件后，可使用直接法或对称正定系统适用的迭代法。当前测试覆盖到 $k = 5$ 的多项式一致性；没有人为阶数上限，但高阶单项式矩阵的条件数会增大，不等于任意高阶都已验证。

表 5.2: 稳定化选项

取值	含义
`"trace"`	默认；迹比缩放的对称稳定化
`"legacy"`	兼容别名，与 trace 相同
`"none"`	只保留投影项，仅诊断使用

5.4 RT0×P0 混合空间

```python
from MosicaFE import Mesh, FESpace

mesh = Mesh.rectangle(nx=16, ny=16)       # triangle mesh
V = FESpace.rt0(mesh)
```

```
5
6  print(V.n_flux, V.n_pressure, V.n_dofs)   # facets, cells, sum
7  print(V.cell_flux_dofs(0))                # local facet -> global flux DOF
8  print(V.facet_signs(0))                   # outward orientation signs
9  print(V.local_flux_mass(0))               # 单元 0 的精确 RT0 质量矩阵 (3x3)
```

Listing 5.3: 示例 5.3：混合空间

5.4.1 空间层给出的两个块

RT0空间与 LagrangeSpace / VEMSpace 的接口是**平行**的，只是把”单个椭圆算子”换成了”鞍点问题的两个块”：

表 5.3: RT0×P0 的矩阵原语（供物理层组合）

方法	形状	含义
`V.flux_mass_matrix()`	(Φ, Φ)	$M_{ij} = \int_K \varphi_i \cdot \varphi_j$，**精确闭式**，无需求积
`V.divergence_matrix()`	(P, Φ)	$B_{K,i} = \int_K \nabla \cdot \varphi_i = \sigma_{K,i}$，恒为 ± 1
`V.pressure_load_vector(f)`	$(P,)$	$\int_K f$
`V.interpolate_pressure(f)`	$(P,)$	f 的单元平均（L^2 投影到 P0）
`V.interpolate_flux(q)`	$(\Phi,)$	$\Phi_f = \int_f q \cdot n_f \, \mathrm{d}s$
`V.evaluate_flux(c,Q,pts)`	(n_q, d)	单元内重构 $q_h = \sum_i F_i \varphi_i$
`V.locate_cell(x)`	int	用重心坐标定位点所在单元

```
1  from MosicaFE import Mesh, FESpace
2  from MosicaFE._compat import bmat
3
4  mesh = Mesh.rectangle(nx=16, ny=16)
5  V = FESpace.rt0(mesh)
6
7  M = V.flux_mass_matrix()          # (Phi,Phi)
8  B = V.divergence_matrix()         # (P,Phi)
9  f = V.pressure_load_vector(src)   # (P,)
10
11 A = bmat([[M, -B.T], [B, None]])  # [[M, -B^T], [B, 0]]
```

Listing 5.4: 示例 5.4：直接使用两个块

注意

RT0 空间**不提供** `stiffness_matrix()` / `mass_matrix()` / `load_vector()`，也不提供 `interpolate()`：它对应的是鞍点问题，而且同时携带两个场，没有”单一算子 / 单一插值”可言。调用这些方法会抛出 `NotImplementedError`，并在消息里指出该用哪个方法（例如 `mass_matrix()` → `flux_mass_matrix()`，`load_vector()` → `pressure_load_vector()`）。日常使用请直接配合 `MixedPoissonProblem`。

5.5 空间工厂 FESpace

```python
from MosicaFE import Mesh, FESpace

mesh = Mesh.rectangle(nx=16, ny=16)

V_fem = FESpace.lagrange(mesh, degree=2)     # P2
V_vem = FESpace.vem(mesh, degree=3)          # VEM k=3
V_rt0 = FESpace.rt0(mesh)                    # RT0 x P0

# by name (used by solve_poisson(element=...))
print(FESpace.available())
V_any = FESpace.create(mesh, family="vem", degree=3)
```

Listing 5.5: 示例 5.4：三种空间族

表 5.4: 字符串形式的”元”名字（`solve_poisson(element=...)` 使用）

名字	空间族	可用网格
`"P1"` / `"P2"`	Lagrange	三角形、四面体
`"Q1"` / `"Q2"`	Lagrange（张量积）	四边形、六面体
`"VEM"` / `"VEM3"`	虚拟元 $k \geq 1$	合格多边形 / 多面体
`"RT0"` / `"RT0-P0"` / `"MIXED"`	RT0×P0	三角形、四面体

第六章　物理问题与组装

6.1　问题接口

所有 PDE 都继承 `BasePhysics`，只需实现一个方法：

```python
class BasePhysics(ABC):
    def __init__(self, space, f=None, g=None, **kwargs):
        self.space = space
        self.mesh = space.mesh
        self.n_dofs = space.n_dofs
        self.f = f or (lambda x: 0.0)      # source term
        self.g = g or (lambda x: 0.0)      # Dirichlet data
        self.boundary_dofs = space.boundary_dofs()

    @abstractmethod
    def assemble(self):
        """return (A, b) with boundary conditions applied"""

    def solve(self, method="auto", **kwargs):
        A, b = self.assemble()
        return Solver.solve(A, b, method=method, **kwargs)
```

Listing 6.1: 示例 6.1：`BasePhysics` 接口

6.2　Poisson 问题

```python
@register_problem("poisson")
class PoissonProblem(BasePhysics):
    def __init__(self, space, f=None, g=None, kappa=1.0, **kwargs):
        super().__init__(space, f=f, g=g, **kwargs)
        self.kappa = kappa

    def assemble(self):
        A = self.space.stiffness_matrix()
        if float(self.kappa) != 1.0:
            A = float(self.kappa) * A
```

```
        b = self.space.load_vector(self.f)
        A, b = self.apply_boundary_conditions(A, b)
        return A, b
```

Listing 6.2: 示例 6.2：Poisson 问题的组装（完整实现）

要点

整个实现只有 8 行，而且它对 **Lagrange 元与虚拟元同时成立**。这正是把” 单元循环” 下沉到 `MosicaFE.spaces` 层之后的收益。

6.3 边界条件

6.3.1 强加 Dirichlet 条件的正确做法

对边界自由度 j 强加 $u_j = g_j$，标准做法是” 消行消列 + 右端项修正”：

(1) 把边界列对内部行的贡献搬到右端项：

$$b_i \leftarrow b_i - \sum_{j \in \mathcal{D}} A_{ij} g_j, \qquad \forall i \notin \mathcal{D}; \tag{6.1}$$

(2) 删去所有涉及 $\mathcal{D}$ 的行与列；

(3) 对 $j \in \mathcal{D}$ 置 $A_{jj} = 1$，$b_j = g_j$。

注意

第 (1) 步在齐次条件 $g = 0$ 时” 看起来” 可以省略，因此很多演示程序都漏掉了它；一旦边界数据非零，解就是错的。MosicaFE 用一个简单而灵敏的测试——**非齐次线性 patch test**——来防止这类错误回归：

$$u(x) = 1 + 2x_1 - \tfrac{1}{2}x_2, \qquad f \equiv 0, \qquad g = u|_{\partial\Omega}, \tag{6.2}$$

此时精确解属于离散空间，数值解应当**精确**（误差 $\sim 10^{-15}$）。修复前的实测误差是 2.875。

6.3.2 其它边界条件

- **Neumann 条件**：向 `load_vector` 的结果中加入 $\int_{\Gamma_N} h\varphi_i \, \mathrm{d}s$ 即可，需要边界 facet 的求积规则；
- **RT0 混合格式**：$p = 0$ 是自然边界条件，无需消去任何自由度（这也是混合格式在流量问题上更” 自然” 的原因）；
- 非齐次 $p = g \neq 0$ 在混合格式下需要额外处理边界通量项，当前版本会显式报错而不是给出错误结果。

6.4 组装算法

Algorithm 1 全局组装（MosicaFE 的 `BaseFESpace._assemble`）

Require: 空间 V、单元局部矩阵生成函数 `local_matrix(cell)`

初始化三元组列表 `rows,cols,vals`
for 每个单元 $c = 0, \dots, n_{\text{cells}} - 1$ **do**
  $\mathcal{D}_c \leftarrow$`get_cell_dofs(c)`
  $K_c \leftarrow$`local_matrix(c)`
  把 $\mathcal{D}_c$ 的行 / 列索引与 K_c 展平后追加到三元组列表
end for
由 COO 三元组构造矩阵（重复下标自动求和）
if 要求对称 **then**
  $A \leftarrow \frac{1}{2}(A + A^\top)$
end if
return A

注意

右端项组装必须使用”逐项累加”语义（`numpy` 的 `np.add.at`）。若直接用 `b[dofs] += local`，当 `dofs` 中出现重复下标时（例如网格里存在重复顶点），`numpy` 的缓冲赋值会丢掉累加，而稀疏矩阵的 COO 组装却会正确求和——两者不一致会造成难以察觉的错误。

6.5 使用 Poisson 问题

```python
import numpy as np
from MosicaFE import Mesh, FESpace, PoissonProblem

def source(x):
    return 2*np.pi**2 * np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

mesh = Mesh.rectangle(nx=32, ny=32)
V = FESpace.lagrange(mesh, degree=2)

problem = PoissonProblem(V, f=source, g=lambda x: 0.0, kappa=1.0)

A, b = problem.assemble()
print("system size:", A.shape, "non-zeros:", A.nnz)

u = problem.solve(method="direct")
print("solution size:", u.size)
```

Listing 6.3: 示例 6.3：组装与求解

6.6 PDE 注册表

```python
from MosicaFE import FESpace, Mesh, available_problems, create_problem, run

print(available_problems())        # ['mixed_poisson', 'poisson', ...]

mesh = Mesh.rectangle(nx=32, ny=32)
problem = create_problem("poisson", FESpace.vem(mesh),
                         f=source, g=lambda x: 0.0)
u = problem.solve()

u2 = run("poisson", mesh, element="VEM", f=source, g=lambda x: 0.0)
```

Listing 6.4: 示例 6.4：按名字创建与求解方程

第七章　线性求解器

7.1　求解器接口

```python
import numpy as np, time
from MosicaFE import FESpace, Mesh, PoissonProblem, Solver

mesh = Mesh.rectangle(nx=32, ny=32)
problem = PoissonProblem(FESpace.lagrange(mesh, 2), f=source,
                         g=lambda x: 0.0)
A, b = problem.assemble()

t0 = time.time(); u_direct = Solver.direct(A, b); t1 = time.time()
u_cg    = Solver.conjugate_gradient(A, b, tol=1e-10)
u_gmres = Solver.gmres(A, b, tol=1e-10, restart=30)
u_bicg  = Solver.bicgstab(A, b, tol=1e-10)

print("direct %.3f s, diff vs CG: %.3e"
      % (t1 - t0, np.linalg.norm(u_direct - u_cg)))
```

Listing 7.1: 示例 7.1：Solver 接口

也可以让问题对象自己选择：

```python
u = problem.solve(method="auto")   # auto/direct/cg/gmres/bicgstab/minres
```

7.2　直接法

- 稀疏矩阵：`scipy.sparse.linalg.spsolve`（内部为稀疏 LU / Cholesky）；
- 稠密矩阵：`numpy.linalg.solve`；
- 复杂度：2D 约 $O(n^{1.5})$，3D 约 $O(n^2)$。

表 7.1: 可用的 Krylov 求解器

方法	适用矩阵	备注
`conjugate_gradient`	对称正定	每步 $O(\text{nnz})$，迭代数 $O(\sqrt{\kappa})$
`gmres`	一般（非对称）	带重启，存储 $O(\text{restart} \times n)$
`bicgstab`	一般	每步两次矩阵向量积，内存小
`minres`	对称不定	例如 RT0 的鞍点系统

7.3 迭代法

注意

共轭梯度法要求施加边界条件后的矩阵对称正定。VEM 的 `trace` 与 `legacy` 使用相同的对称稳定化；`none` 不能保证控制投影核。
鞍点系统请用直接法或 MINRES。

7.4 求解器选择指南

表 7.2: 求解器选择建议

问题规模	矩阵类型	推荐求解器	备注
$< 2 \times 10^4$ 自由度	任意	直接法	精确、无需调参
$> 2 \times 10^4$	对称正定	CG	建议配预处理
$> 2 \times 10^4$	对称不定	MINRES 或直接法	鞍点问题
$> 2 \times 10^4$	非对称	GMRES / BiCGSTAB	需要重启参数

```python
from MosicaFE import Solver

print(Solver.recommend(1000, symmetric=True))     # 'direct'
print(Solver.recommend(10**6, symmetric=True))    # 'cg'
print(Solver.recommend(10**6, symmetric=False))   # 'gmres'
```

Listing 7.2: 示例 7.2：自动选择

第八章　误差分析与收敛性

8.1　误差范数

$$\|e\|_{L^2} = \left(\int_\Omega |u - u_h|^2 \, \mathrm{d}x \right)^{1/2}, \qquad \|e\|_{L^2}^2 \approx \sum_{K} \sum_{q} w_q \, |u(x_q) - u_h(x_q)|^2 \, |\det J_K|, \tag{8.1}$$

$$|e|_{H^1} = \left(\int_\Omega |\nabla u - \nabla u_h|^2 \, \mathrm{d}x \right)^{1/2}, \qquad \|e\|_{L^\infty} = \max_{x \in \Omega} |u(x) - u_h(x)|. \tag{8.2}$$

```python
import numpy as np

def exact(x):
    return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

def exact_grad(x):
    return np.array([np.pi*np.cos(np.pi*x[0])*np.sin(np.pi*x[1]),
                     np.pi*np.sin(np.pi*x[0])*np.cos(np.pi*x[1])])

u = problem.solve()
l2   = problem.compute_error(u, exact, "L2")
linf = problem.compute_error(u, exact, "Linf")
h1   = problem.compute_error(u, exact, "H1", exact_grad=exact_grad)

print("%.6e %.6e %.6e" % (l2, h1, linf))

norms = problem.compute_errors(u, exact, exact_grad)
```

Listing 8.1: 示例 8.1：计算三种范数

注 8.1. 虚拟元的 `evaluate` 使用 $\Pi_k^0 u_h$，`evaluate_gradient` 使用 $\nabla \Pi_k^\nabla u_h$。因此报告的是可计算的投影误差；不能直接称为未知虚拟函数的精确误差。`H1` 表示梯度误差的半范数，`Linf` 是离散采样最大误差。高阶解向量包含矩自由度，不可直接对全部 `u` 取最大值作为函数最大值。

8.2　收敛性研究框架

```python
from MosicaFE import FESpace, Mesh, PoissonProblem, convergence_study
from MosicaFE.postprocess import estimate_convergence_rate

def mesh_factory(level):
    n = 4 * 2**level
    return Mesh.rectangle(nx=n, ny=n)

def problem_factory(mesh):
    V = FESpace.lagrange(mesh, degree=2)
    return PoissonProblem(V, f=source, g=lambda x: 0.0)

h, errors = convergence_study(mesh_factory, problem_factory,
                              exact, exact_grad,
                              refinement_levels=5, plot=True,
                              filename="conv_p2.png")

print(estimate_convergence_rate(h, errors["L2"], use_last=3))   # ~3.0
```

Listing 8.2: 示例 8.2：自动收敛性研究

8.3 收敛阶估计

由两层级误差

$$\text{rate} = \frac{\log(e_1/e_2)}{\log(h_1/h_2)}. \tag{8.3}$$

更稳健的做法是对 $\log e$–$\log h$ 做最小二乘拟合（`estimate_convergence_rate(h,errors)` 就是这样实现的），并且可以用 `use_last=k` 只取最后 k 层以逼近渐近阶。

8.4 实测验证结果

下表保留 FEM/RT0 的既有验证记录。新版任意阶 VEM 的可追溯结果见第 10.3.1 节。

复现方式：

```
& "D:\anaconda\envs\fealpy\python.exe" MosicaFE\tests\run_checks.py
```

表 8.1: 各方法的实测收敛阶（制造解，网格逐次加密）

元	网格	实测 L^2	实测 H^1	理论 (L^2/H^1)
P1	三角形	1.98	0.99	2 / 1
P2	三角形	3.00	1.99	3 / 2
Q1	四边形	2.00	1.00	2 / 1
Q2	四边形	3.00	2.00	3 / 2
P1	四面体	1.95	0.98	2 / 1
Q1	六面体	2.01	1.00	2 / 1
RT0×P0	三角形（压力）	1.00	—	1
RT0×P0	三角形（通量）	1.00	—	1

第九章　可视化

9.1　可视化工具一览

表 9.1: `MosicaFE.postprocess` 中的绘图函数

函数	用途
`plot_solution(u,mesh,V,...)`	画解。1D 画曲线；2D 画等值线填充或三维曲面；3D 画 $z = \text{const}$ 切片。多边形网格自动从形心做扇形三角化
`plot_mesh(mesh,...)`	画网格（3D 画中间切片）
`plot_error(u,mesh,space, exact,...)`	画逐点误差 $\lvert u_h - u \rvert$
`plot_convergence(h,errors, ...)`	log–log 收敛曲线，并叠加 $O(h^k)$ 参考斜率
`plot_pressure_2d(mesh,P, ...)`	RT0 的分片常数压力
`plot_flux_2d(mesh,Q,...)`	通量箭头图
`compare_solutions(entries, ...)`	多组解并排比较

通用参数：`title=`、`filename=`（保存 PNG）、`show=`（弹窗）、`dpi=`、`ax=`（画到已有坐标轴上以便拼图）。

9.2　画解

```python
from MosicaFE import Mesh, FESpace, PoissonProblem
from MosicaFE.postprocess import plot_solution, plot_mesh, plot_error

# --- 2D contour ---
mesh = Mesh.rectangle(nx=32, ny=32)
V = FESpace.lagrange(mesh, 2)
u = PoissonProblem(V, f=source, g=lambda x: 0.0).solve()
```

```
plot_solution(u, mesh, V, title="FEM P2", filename="sol2d.png")

# --- 2D surface ---
plot_solution(u, mesh, V, mode="surface", filename="sol_surface.png")

# --- polygon mesh + VEM solution ---
mp = Mesh.voronoi_polygon(n_points=200, seed=3)
Vv = FESpace.vem(mp)
uv = PoissonProblem(Vv, f=source, g=lambda x: 0.0).solve()
plot_solution(uv, mp, Vv, title="VEM on polygons",
              filename="sol_vem.png")
plot_error(uv, mp, Vv, exact, filename="err_vem.png")

# --- 3D slice ---
m3 = Mesh.box(nx=8, ny=8, nz=8)
V3 = FESpace.lagrange(m3, 1)
u3 = PoissonProblem(V3, f=source3d, g=lambda x: 0.0).solve()
plot_solution(u3, m3, V3, z_slice=0.5, filename="sol3d_slice.png")
plot_mesh(m3, filename="mesh3d.png")
```

Listing 9.1: 示例 9.1：2D 与 3D 解的绘制

9.3 画收敛曲线

```
from MosicaFE.postprocess import plot_convergence

plot_convergence(h, errors, filename="convergence.png",
                 reference_rates=(1, 2, 3))
```

Listing 9.2: 示例 9.2：收敛曲线

注意

所有绘图函数在没有安装 `matplotlib` 时只会打印提示并返回 `None`，不会抛异常，因此批量计算的脚本可以放心调用。

第十章　完整示例

10.1　示例一：两行求解 Poisson 方程

```python
import numpy as np
from MosicaFE import Mesh, solve_poisson

source = lambda x: 2*np.pi**2 * np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

u = solve_poisson(Mesh.rectangle(nx=32, ny=32), element="P2", f=source)
print(u.size)          # 4225
```

Listing 10.1: 示例 10.1：一行式求解

10.2　示例二：换一种元（同一段代码，三种方法）

```python
import numpy as np
from MosicaFE import FESpace, Mesh, MixedPoissonProblem, PoissonProblem

def exact(x):
    return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

def grad(x):
    return np.array([np.pi*np.cos(np.pi*x[0])*np.sin(np.pi*x[1]),
                     np.pi*np.sin(np.pi*x[0])*np.cos(np.pi*x[1])])

def source(x):
    return 2*np.pi**2 * exact(x)

def zero(x):
    return 0.0

mesh = Mesh.rectangle(nx=32, ny=32)

# (a) classic FEM: P2 Lagrange
V = FESpace.lagrange(mesh, degree=2)
```

```
p = PoissonProblem(V, f=source, g=zero)
u = p.solve()
print("FEM P2 :", p.compute_errors(u, exact, grad))

# (b) VEM: only the space changes
V = FESpace.vem(mesh, degree=3)
p = PoissonProblem(V, f=source, g=zero)
u = p.solve()
print("VEM k=3:", p.compute_errors(u, exact, grad))

# (c) RT0 x P0 mixed: space + problem class
V = FESpace.rt0(mesh)
p = MixedPoissonProblem(V, f=source)
Q, P = p.solve()
print("RT0-P0 :", p.compute_errors((Q, P), exact, lambda x: -grad(x)))
```

Listing 10.2: 示例 10.2：FEM / VEM / RT0 共用同一套流程

当前程序会打印各方法的误差；新版 VEM 的细网格结果见下节。

10.3 示例三：任意阶 VEM 的二维与三维主程序

仍采用 FEM 主程序的阅读顺序：问题数据、选网格、选元与求解器、可视化。方程为 $-\Delta u = f$，区域为 $(0,1)^d$，精确解为 $u = \prod_{j=1}^{d} \sin(\pi x_j)$，$f = d\pi^2 u$，边界为零。以下两个程序可分别保存到 `examples` 目录运行；代码库中的完整版本是 `examples/vem_poisson.py` 和 `examples/vem_poisson_3d.py`，另含网格图、误差与 JSON 输出。

```
import os
import sys
import numpy as np

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.dirname(os.path.dirname(HERE)))
from MosicaFE import Mesh, plot_solution, solve_poisson

OUT = os.path.join(HERE, "output", "vem_2d")
os.makedirs(OUT, exist_ok=True)

# Problem data.
def exact(x):
    return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])

def source(x):
    return 2*np.pi**2*exact(x)

```

```
def main(n_points=1024, degree=2):
    # Mesh.
    mesh = Mesh.voronoi_polygon(n_points=n_points, seed=42)
    # Element, solver, and solve.
    out = solve_poisson(mesh, element="VEM", degree=degree,
                        f=source, g=lambda x: 0.0,
                        solver="direct", return_all=True)
    u, V = out["u"], out["space"]
    # Visualization.
    plot_solution(u, mesh, V, title=f"VEM{degree} (2D)",
                  filename=os.path.join(OUT, f"solution_k{degree}.png"))
    values = V.nodal_values(u)
    print("DOFs:", u.size, "vertex range:", values.min(), values.max())

if __name__ == "__main__":
    main()
```

Listing 10.3: 二维独立主程序

```
import os
import sys
import numpy as np

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.dirname(os.path.dirname(HERE)))
from MosicaFE import Mesh, plot_solution, solve_poisson

OUT = os.path.join(HERE, "output", "vem_3d")
os.makedirs(OUT, exist_ok=True)

# Problem data.
def exact(x):
    return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1]) * np.sin(np.pi*x[2])

def source(x):
    return 3*np.pi**2*exact(x)

def main(n_points=192, degree=2):
    # Mesh.
    mesh = Mesh.voronoi_polyhedron(n_points=n_points, seed=42)
    # Element, solver, and solve.
    out = solve_poisson(mesh, element="VEM", degree=degree,
                        f=source, g=lambda x: 0.0,
                        solver="direct", return_all=True)
    u, V = out["u"], out["space"]
```

```
    # Visualization.
    plot_solution(u, mesh, V, title=f"VEM{degree} (3D)", z_slice=0.5,
                  filename=os.path.join(OUT, f"solution_k{degree}.png"))
    values = V.nodal_values(u)
    print("DOFs:", u.size, "vertex range:", values.min(), values.max())

if __name__ == "__main__":
    main()
```

Listing 10.4: 三维独立主程序

只改 `degree` 就能切换阶数，`main(n_points=...,degree=3)` 可运行三阶。二维默认 1024 个多边形，三维默认 192 个多面体，输出目录分开。二维解图采用顶点插值；三维 `z_slice=0.5` 按真实平面切割积分四面体，在截面点计算 L^2 投影值，不能理解为挑选附近顶点作图。

10.3.1 细网格结果与误差图

下表直接取自 `examples/output/vem_2d/summary.json` 与 `examples/output/vem_3d/summary.json`，随机种子均为 42。这里报告 $\|u - \Pi_k^0 u_h\|_{L^2}$ 和 $\|\nabla u - \nabla \Pi_k^\nabla u_h\|_{L^2}$；$H^1$ 一列是半范数误差。

表 10.1: 保存的二维与三维 VEM 细网格结果

d	N	k	自由度	L^2	H^1
2	256	2	1539	1.952×10^{-4}	1.328×10^{-2}
2	1024	2	6147	2.548×10^{-5}	3.378×10^{-3}
2	1024	3	11268	4.038×10^{-7}	6.825×10^{-5}
3	96	2	2361	7.294×10^{-3}	1.781×10^{-1}
3	192	2	4977	4.015×10^{-3}	1.212×10^{-1}
3	192	3	10527	9.961×10^{-4}	1.919×10^{-2}

误差图将各维度分开展示。每组只有两个非嵌套网格的二阶结果，三阶仅有细网格点，因此用于观察加密和提阶效果，不据此宣称渐近收敛阶。在常规网格正则性、解的光滑性、稳定化及求积精度条件成立时，预期 H^1 误差为 $O(h^k)$，L^2 误差为 $O(h^{k+1})$；当前高阶单项式基的数值条件数也会影响实际表现。

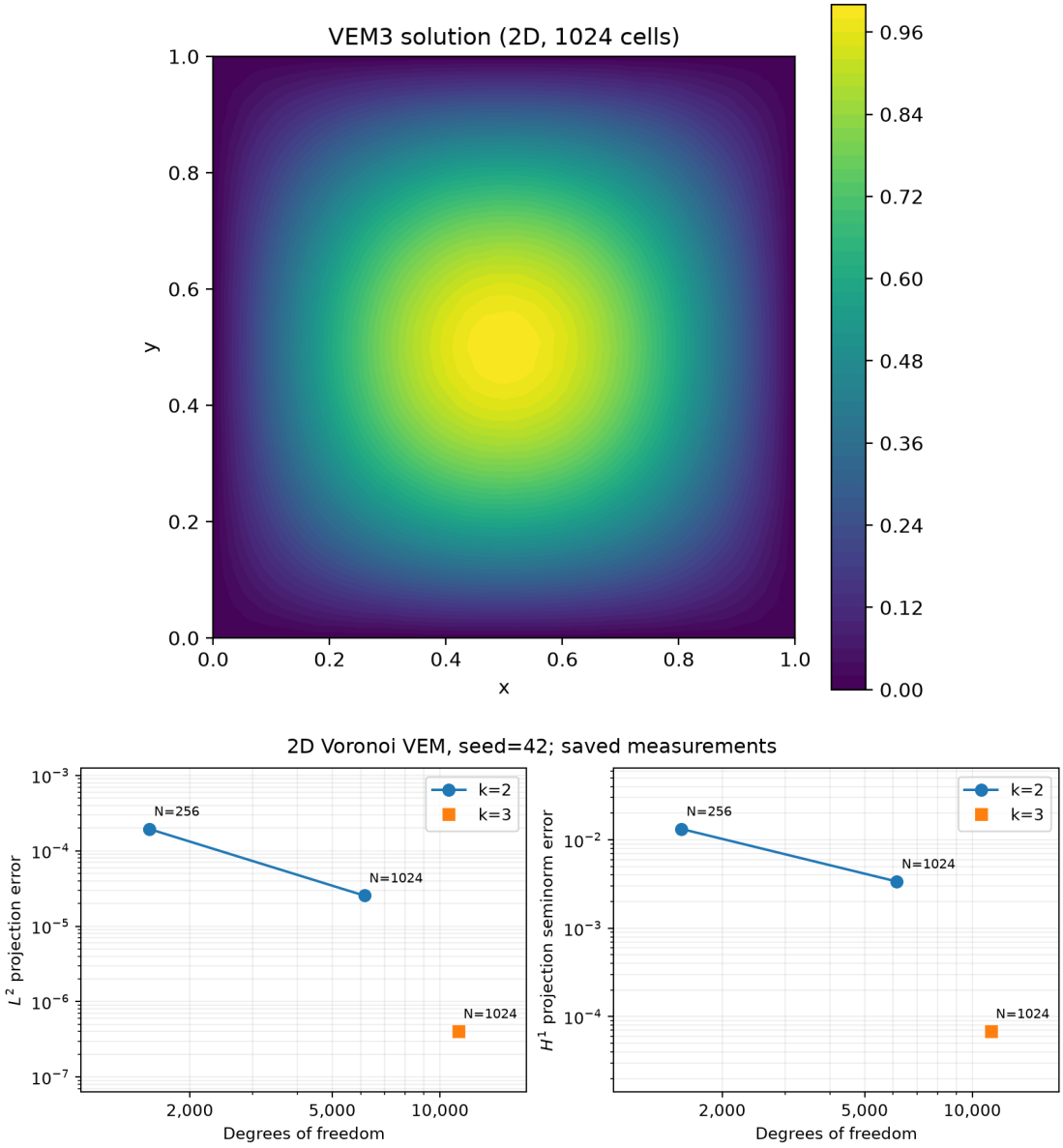


图 10.1: 2 维三阶 VEM 解图（1024 个单元）及保存结果的误差比较。

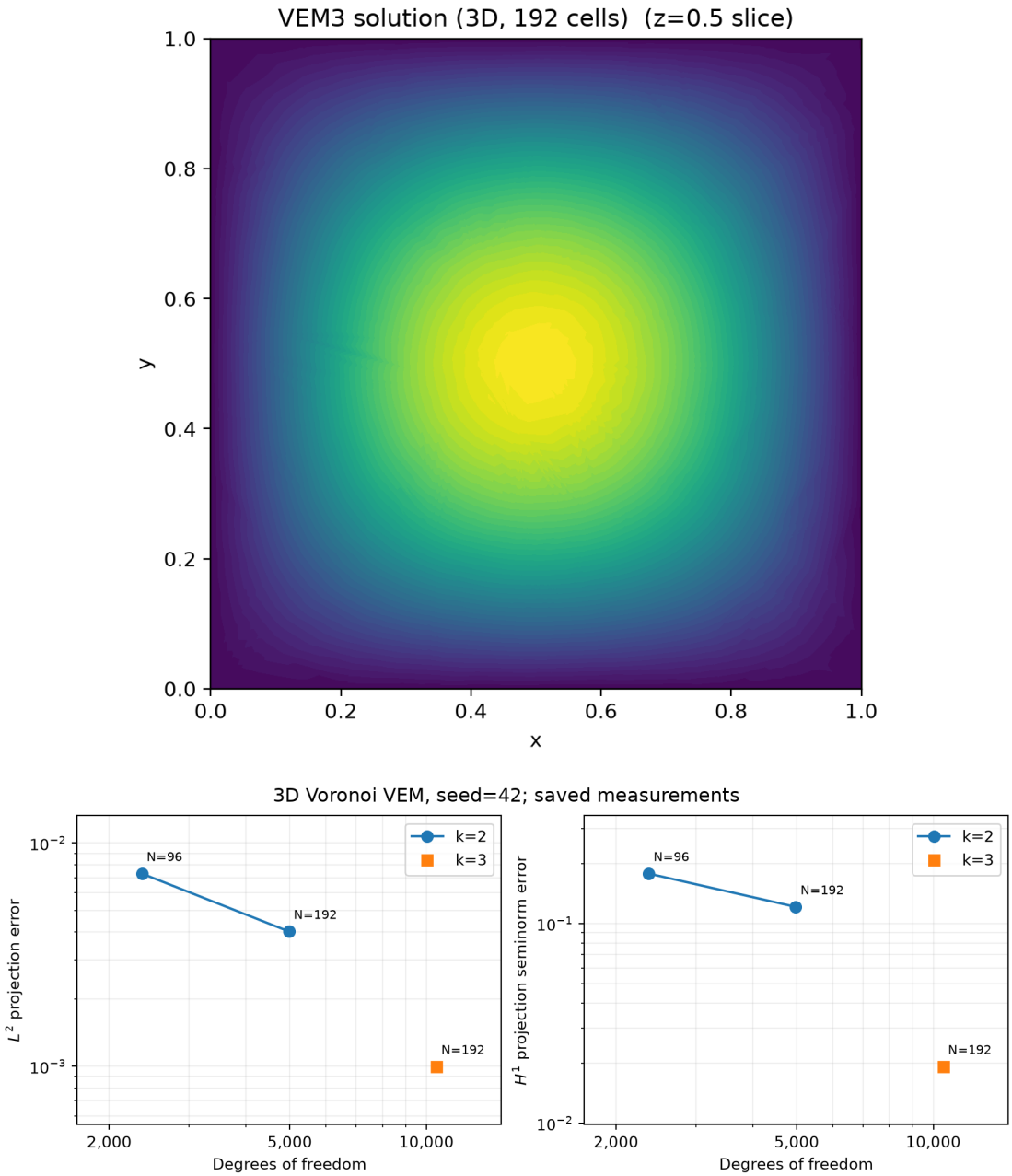


图 10.2: 3 维三阶 VEM 解图（192 个单元）及保存结果的误差比较。解图为 $z = 0.5$ 真实截面。

在项目根目录复现默认细网格算例，并检查实现：

```
python examples/vem_poisson.py
python examples/vem_poisson_3d.py
python tests/run_tests.py
```

10.4 示例四：自定义新 PDE（Helmholtz）

```python
import numpy as np
from MosicaFE import FESpace, Mesh
from MosicaFE.physics.base import BasePhysics
from MosicaFE.physics.registry import register_problem

@register_problem("helmholtz")
class HelmholtzProblem(BasePhysics):
    """-Laplacian(u) - k^2 u = f, weak form: (grad u, grad v) - k^2 (u, v) = (f, v)"""

    def __init__(self, space, f=None, g=None, k=1.0, **kwargs):
        super().__init__(space, f=f, g=g, **kwargs)
        self.k = k

    def assemble(self):
        A = self.space.stiffness_matrix() - self.k**2 * self.space.mass_matrix()
        b = self.space.load_vector(self.f)
        return self.apply_boundary_conditions(A, b)

# use it with any space family
k = 1.0
exact = lambda x: np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
source = lambda x: (2*np.pi**2 - k**2) * exact(x)

mesh = Mesh.rectangle(nx=32, ny=32)
V = FESpace.vem(mesh)                          # works with VEM too
problem = HelmholtzProblem(V, f=source, g=lambda x: 0.0, k=k)
u = problem.solve(method="direct")
print("max error:", np.abs(u - V.interpolate(exact)).max())
```

Listing 10.5: 示例 10.4：新增一门方程只需十几行

输出：`max error: 9.310e-04`。

10.5 示例五：反应扩散方程

```python
import numpy as np
from MosicaFE import FESpace, Mesh, ReactionDiffusionProblem

c = 10.0
exact = lambda x: np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
source = lambda x: (2*np.pi**2 + c) * exact(x)

mesh = Mesh.rectangle(nx=32, ny=32)
V = FESpace.lagrange(mesh, degree=2)
problem = ReactionDiffusionProblem(V, f=source, g=lambda x: 0.0,
                                   reaction_coeff=c)
u = problem.solve()
print(problem.compute_error(u, exact, "L2"))     # ~6.8e-05
```

Listing 10.6: 示例 10.5：内置的反应扩散问题

第十一章　进阶话题

11.1　精度陷阱与经验教训

把三套程序整合成一个库的过程中，实测暴露了四类问题。它们都不是” 数值精度的小差别”，而是会给出错误结果或让收敛阶崩掉的结构性错误。记录在此，既是为了说明 MosicaFE 为什么这么写，也供读者在自己的代码里自查。

11.1.1　陷阱一：Dirichlet 消去漏掉右端项修正

症状：齐次边界一切正常，非齐次边界解错。

诊断：用线性 patch test（式(6.2)）。精确解属于离散空间时，数值解必须精确。

结论：见式(6.1)，第 (1) 步不可省。MosicaFE 把这条测试写进了 `tests/test_core_spaces.py`。

11.1.2　陷阱二：张量积单元的求积阶不足

症状：三维六面体 Q1 的 L^2 收敛阶算出来是 -0.11。

诊断：Q1 刚度矩阵的被积函数在每个方向上是 2 次多项式，而每个方向只用了 1 个 Gauss 点时只能精确到 1 次，欠积分导致单元矩阵不是正定的、能量被系统性低估。

结论：张量积单元的求积阶取 `max(2,2*p)`（每方向 p 个点起步），见表格 4.5 后的注意事项。

11.1.3　陷阱三：高阶投影的常数规范化

梯度 Gram 矩阵的常数行是零，不能直接求逆。当前实现按式(3.10) 替换常数行：$k = 1$ 用顶点均值，$k \geq 2$ 用真实单元均值。应对 $\mathcal{P}_k$ 检查 $P_\nabla D = I$、$P_0 D = I$，而不只检查一次函数。

11.1.4　陷阱四：把高阶积分矩当作点值

矩自由度是 $|E|^{-1}\int_E u m_\alpha$，不能用实体中心的函数值代替。载荷与质量必须按式(3.15) 用 L^2 投影构造；常函数检查使用 `V.interpolate` 计算常函数自由度，不能假设所有自由度均为 1。归一化面矩也必须用于非零 Dirichlet 数据。

11.1.5 陷阱五：numpy 的重复下标赋值

症状：`b[dofs] += local` 与稀疏矩阵组装结果不一致，质量矩阵的行和与载荷向量对不上。

诊断：`dofs` 中出现重复下标时（例如网格里有重复顶点），`numpy` 的缓冲赋值只保留最后一次，而 COO 组装会求和。

结论：一律使用 `numpy.add.at`。

11.2 网格细化策略

- **均匀加密**：`mesh.refine_uniform()`，三角形单元数 ×4、六面体 ×8；
- **自适应加密（规划中）**：基于残差型误差指示子

$$\eta_K = h_K \|f + \Delta u_h\|_{L^2(K)} + h_K^{1/2} \|[\partial_n u_h]\|_{L^2(\partial K)} \tag{11.1}$$

  标记 $\eta_K > \theta \max_K \eta_K$ 的单元细分。MosicaFE 的 `Mesh` 支持变长单元，因此局部加密在数据结构上是可行的，缺的是 refinement 的实现。

11.3 性能与向量化

(1) 组装阶段累计 COO 三元组，最后一次转成 CSR；

(2) 参考单元上的基函数求值按" 求积点" 缓存（`LagrangeSpace._quadrature_data`），避免逐单元重复计算；

(3) 对同一网格上多种物理问题，`stiffness_matrix()` 的结果可以复用；

(4) 大规模问题建议改用迭代法；

(5) 若要进一步提速，可以把单元循环用 `numba` 或" 按单元类型分块 + 批量张量运算" 的方式向量化。

11.4 与其它库的关系

MosicaFE 刻意保持" 小、可读、可改"：

- 与 FEniCS / Firedrake 相比，MosicaFE 不是通用有限元平台，而是**教学与研究用的可读实现**，代价是只支持结构化网格生成与有限几类单元；
- 与 `fealpy` 相比，MosicaFE 更聚焦于"FEM / VEM / RT0三种离散方式的横向对比"；
- 与原始三套脚本相比，MosicaFE 把它们变成同一套接口、统一了网格与求解器、并补齐了验证。

第十二章　扩展框架

12.1　新增一种单元

步骤：

(1) 新建 MosicaFE/spaces/yourspace.py，继承 BaseFESpace；

(2) 实现 _build_dof_map():建立单元局部自由度到全局自由度的映射,并填写 n_dofs、dof_support、dof_is_interior；

(3) 实现 stiffness_matrix()、mass_matrix()、load_vector(f)；

(4) 实现 interpolate(f) 与 evaluate(cell,u,points)；

(5) 在 spaces/factory.py 的 FESpace.FAMILIES 中注册；

(6) 在 tests/ 中补上 patch test 与收敛阶测试。

```python
from MosicaFE.spaces.base import BaseFESpace

class MySpace(BaseFESpace):
    family = "myspace"
    has_explicit_basis = True

    def _build_dof_map(self):
        # fill self.dof_map, self.n_dofs, self.dof_support, self.dof_is_interior
        ...

    def reference_nodes(self, cell_idx):
        # reference coordinates of local DOFs
        ...

    def eval_basis(self, xi):
        ...

    def eval_basis_grad(self, xi):
        ...

```

```
    def stiffness_matrix(self):
        return self._assemble(self._local_stiffness, symmetric=True)

    def mass_matrix(self):
        return self._assemble(self._local_mass, symmetric=True)

    def load_vector(self, f):
        return self._assemble_vector(lambda c: self._local_load(c, f))

    def interpolate(self, f):
        return self._evaluate_function(f, self.dof_coords)
```

Listing 12.1: 示例 12.1：显式基函数空间的骨架

要点

如果新单元的基函数是显式的（例如更高次的 Lagrange、DG、或张量积的其它族），可以参考 `spaces/lagrange.py`：它已经实现了通用的”参考单元求值 + 逐单元变换”组装循环，通常只需要提供 `eval_basis` 与自由度布局。

12.2 新增一门方程

```
from MosicaFE.physics.base import BasePhysics
from MosicaFE.physics.registry import register_problem

@register_problem("my_pde")
class MyProblem(BasePhysics):
    """Weak form: a(u, v) = l(v)."""

    def __init__(self, space, f=None, g=None, **kwargs):
        super().__init__(space, f=f, g=g, **kwargs)
        # store problem specific data here

    def assemble(self):
        K = self.space.stiffness_matrix()
        M = self.space.mass_matrix()
        A = K + 3.0 * M                        # example combination
        b = self.space.load_vector(self.f)
        return self.apply_boundary_conditions(A, b)
```

Listing 12.2: 示例 12.2：新 PDE 模板

注册之后，`available_problems()` 会包含`"my_pde"`，并且 `create_problem("my_pde", V,...)` 与 `run("my_pde",mesh,element="VEM",...)` 立刻可用。

12.3 新增一种混合格式（以 RT0×P0 为模板）

混合格式（鞍点问题）与上面的”椭圆算子”套路不同，但分工是一样的：**空间算出单元矩阵块，物理层只做拼装**。RT0×P0 在库内就是按这个模板实现的，可以照抄：

(1) 如果需要新的面自由度布局，先在 `core/facets.py` 拿到面拓扑（全局面编号、`cell_facets`、`signs`）；

(2) 在空间里实现”块”接口：本例是 `flux_mass_matrix()`（(Φ,Φ)）与 `divergence_matrix()`（(P,Φ)），外加 `pressure_load_vector(f)`；

(3) 在物理层用 `bmat`（来自 `MosicaFE._compat`）把块拼成鞍点矩阵：`bmat([[M,-B.T],[B, None]])`；

(4) 求解方式：`"direct"`、`"schur"`（静力凝聚）或 `Solver.minres`（对称不定）；

(5) 误差分析放进 `postprocess/error.py`，参照 `mixed_error_norms`（压力 + 通量各一套范数）。

要点

`physics` 层只有 6 行拼装代码（见 `physics/mixed_poisson.py` 的 `assemble`），而单元几何与数值积分全部留在 `spaces/rt0.py`。因此换一套混合元（例如 BDM1、或 Stokes 的 $Q_2 \times Q_1$ 配对）时，只需要提供新的块，物理层与后处理可以原样复用。

12.4 新增一种网格

(1) 在 `core/mesh.py` 中新增一个类方法，返回 `Mesh(nodes,elements,element_type, name=...)`；

(2) 2D 单元会自动按带符号面积纠正为逆时针；3D 四面体会按体积纠正取向；

(3) 如果 3D 单元不是四面体 / 六面体，请通过 `faces=...` 提供每个单元的面表（局部顶点编号的列表）。

```python
import numpy as np
from MosicaFE import Mesh

# a two-element hexagonal domain (arbitrary polygons are fine)
nodes = np.array([[0.0, 0.0], [1.0, 0.0], [1.5, 0.9],
                  [0.5, 1.5], [-0.5, 0.9], [0.5, 0.6]])
elements = [[0, 1, 2, 5], [0, 5, 3, 4],
            [1, 2, 3, 5], [0, 3, 4, 5]]

mesh = Mesh.from_arrays(nodes, elements, boundary_nodes=[0, 1, 2, 3, 4])
print(mesh.summary())
```

Listing 12.3: 示例 12.3：自定义单元形状

12.5 开发流程与测试要求

建议的流程：

(1) 先写**失败**的测试（patch test、收敛阶、对称性等）；

(2) 再实现功能；

(3) 跑 `tests/run_tests.py` 与 `tests/run_checks.py`；

(4) 更新 `progress/` 下的进度与验证记录。

每个新特性至少要覆盖：

- 单元级：基函数的 Kronecker 性质、单位分解、单元矩阵对称性/正定性；
- 空间级：全局矩阵的对称性、常数核、M 行和与 $b(f \equiv 1)$ 的一致性；
- 问题级：patch test（线性精确解必须精确）、收敛阶；
- 端到端：`solve_poisson` / `run` 一行接口可用。

第十三章　常见问题与排查

13.1　常见问题

13.1.1　导入错误

现象：`ModuleNotFoundError: No module named 'MosicaFE'`。

原因：MosicaFE 在 `D:\mosica\MosicaFE`，需要把**父目录** `D:\mosica` 加入模块路径。

```
# option 1: install in editable mode
cd D:\mosica\MosicaFE && python -m pip install -e .

# option 2: extend PYTHONPATH
$env:PYTHONPATH = "D:\mosica"
```

13.1.2　收敛阶不对

排查顺序：

(1) 先跑 patch test（第六章的式(6.2)）。若不通过，问题一定在组装或边界处理；

(2) 检查求积阶是否足够（张量积单元尤其容易欠积分）；

(3) 检查h 的定义是否与网格加密一致；

(4) 检查精确解是否足够光滑（制造解最好用三角函数或多项式）；

(5) 对 VEM，检查稳定化选项是否为`"trace"`。

13.1.3　迭代法不收敛

- 先用直接法验证系统本身可解；
- CG 要求矩阵对称正定：鞍点问题必须换 MINRES 或直接法；
- 增大 `maxiter`、放宽 `tol`；
- 条件数很大时考虑缩放或预处理。

13.1.4 内存不足

- 确认在使用稀疏矩阵（安装了 `scipy`）；
- 三维问题网格数增长很快：n^3 个节点对应约 $6n^3$ 个四面体；
- 改用迭代法可避免直接法的填充（fill-in）。

13.2 常见问答

问：怎样选择多项式次数？ 答：绝大多数情形 P2 / Q2 的精度/开销比最好；先用 P1 做快速验证，解很光滑时再上二次元。VEM 支持任意正整数 k，用 `degree=k` 选择；高阶需留意矩阵条件数。

问：什么时候用迭代法？ 答：自由度少于约 2×10^4 时直接法最省心；更大规模改用 CG（对称正定）或 GMRES（一般）。

问：VEM 与 FEM 到底差在哪？ 答：VEM 通过多项式投影和稳定化使用隐式基函数，可直接处理合格的多边形、多面体及不同面数的单元。三角形一阶是与 P1 接近的特殊情形；高阶 VEM 含积分矩，不能直接等同于同阶 Lagrange 元。其几何条件见第 4.1.2 节。

问：为什么 RT0 的解返回两个数组？ 答：混合格式同时求解通量 q_h 与压力 p_h。`Q[c,i]` 是单元 c 第 i 个面（按" 不含顶点 i" 编号）上的外法向通量，`P[c]` 是该单元的常数压力。

问：RT0 需要安装或接入外部程序吗？ 答：不需要。RT0×P0 已经是 MosicaFE 自己的方法：显式基函数 $\varphi_i = (x - a_i)/(d|K|)$ 与精确质量矩阵在 `spaces/rt0.py`，面拓扑与定向在 `core/facets.py`，鞍点组装在 `physics/mixed_poisson.py`，误差范数在 `postprocess/error.py`。库的依赖仍然只有 numpy（scipy / matplotlib 可选），`import MosicaFE` 之后 `FESpace.rt0(mesh)` 直接可用。

问：能求解时间相关问题吗？ 答：当前版本只处理稳态椭圆问题。时间相关问题建议用" 线方法"：空间离散得到 $M\dot{u} + Au = b$，时间方向自行用显式/隐式格式推进。`mass_matrix()` 已经提供。

问：能求解非线性问题吗？ 答：框架面向线性问题。非线性问题建议在外层写 Newton 迭代，每一步用 MosicaFE 组装雅可比并求解。

问：怎样把结果导出到 ParaView？ 答：用 `space.nodal_values(u)` 取得节点值，配合 `mesh.nodes` 与 `mesh.elements` 自行写 `.vtu` 或 `.vtk`；也可以在 `postprocess` 中新增导出函数。

附录 A　API 速查

A.1　顶层模块 MosicaFE.

名字	说明
Mesh	网格对象与工厂方法
FESpace	空间工厂：lagrange / vem / rt0 / create
FacetTopology, build_facet_topology	面拓扑与定向（$H(\mathrm{div})$ 空间）
QuadratureRule	参考单元求积规则
cell_quadrature(...) / simplex_quadrature(...)	物理单元 / 任意单纯形上的求积点与权重
BaseFESpace	空间基类
LagrangeSpace / VEMSpace / RT0Space	三种空间
rt0_flux_basis, rt0_mass_matrices	RT0 显式基函数与精确质量矩阵
BasePhysics	物理问题基类
PoissonProblem	$-\nabla\cdot(\kappa\nabla u)=f$
MixedPoissonProblem	RT0×P0 混合形式
ReactionDiffusionProblem	$-\Delta u+cu=f$
register_problem, create_problem, available_problems	PDE 注册表
Solver	线性求解器（direct / cg / gmres / bicgstab / minres / auto）
compute_error / compute_errors	误差范数
mixed_error_norms	RT0×P0 的压力 / 通量误差范数
convergence_study	收敛性研究
estimate_convergence_rate	收敛阶估计
plot_solution / plot_mesh / plot_error	可视化
plot_convergence / plot_pressure_2d / plot_flux_2d	可视化
solve_poisson	一行式 Poisson 求解
run	按名字求解任意已注册 PDE

A.2 Mesh

方法 / 属性	说明
`Mesh.interval(a,b,nx)`	一维区间网格
`Mesh.rectangle(...)`	二维矩形网格（`triangle/quad`）
`Mesh.box(...)`	三维长方体网格（`tet/hex`）
`Mesh.pentagon(nx,ny)`	五边形网格
`Mesh.voronoi_polygon(...)`	Voronoi 多边形网格
`Mesh.voronoi_polyhedron(...)`	Voronoi 多面体网格
`Mesh.from_arrays(...)`	自定义网格
`mesh.refine_uniform()`	均匀加密
`mesh.get_mesh_size()`	最大单元直径 h
`mesh.get_cell_vertices(i)`	单元顶点坐标
`mesh.get_cell_measure(i)`	单元面积 / 体积
`mesh.get_cell_diameter(i)`	单元直径
`mesh.get_cell_centroid(i)`	单元形心
`mesh.boundary_nodes`	边界顶点编号
`mesh.boundary_facets` / `mesh.facets`	facet 拓扑
`mesh.facet_topology`	`FacetTopology`：全局面编号、共享面、外法向符号
`mesh.summary()`	一行摘要

A.3 FESpace 与空间方法

方法	说明
`FESpace.lagrange(mesh, degree)`	连续 Lagrange（P1/P2/Q1/Q2）
`FESpace.vem(mesh,degree=k)`	增强型虚拟元（正整数 k）
`FESpace.rt0(mesh)`	RT0×P0 混合空间
`FESpace.create(mesh,family, degree)`	按名字创建
`FESpace.available()`	可用空间族
`V.n_dofs`	自由度个数
`V.get_cell_dofs(i)`	单元局部 → 全局自由度
`V.get_dof_coordinates(k)`	自由度坐标
`V.boundary_dofs()`	Dirichlet 自由度

方法	说明
`V.eval_basis(xi)` / `V.eval_basis_grad(xi)`	参考单元基函数
`V.stiffness_matrix()`	刚度矩阵
`V.mass_matrix()`	质量矩阵
`V.load_vector(f)`	载荷向量
`V.interpolate(f)`	插值
`V.evaluate(c,u,pts)`, `V.evaluate_gradient(...)`	单元内重构求值
`V.nodal_values(u)`	节点值（可视化用）
`V.flux_mass_matrix()`	RT0：(Φ,Φ) 块（精确闭式质量矩阵）
`V.divergence_matrix()`	RT0：(P,Φ) 块（散度，恒为 ±1）
`V.pressure_load_vector(f)`	RT0：压力方程右端项 $\int_K f$
`V.interpolate_pressure(f)` / `V.interpolate_flux(q)`	RT0：两个场各自的插值
`V.evaluate_flux(c,Q,pts)`	RT0：单元内重构 q_h
`V.locate_cell(x)`	RT0：定位点所在单元
`V.n_flux` / `V.n_pressure` / `V.topology`	RT0：自由度规模与面拓扑

A.4 物理问题

方法	说明
`PoissonProblem(V,f,g,kappa)`	构造
`problem.assemble()`	返回 (A,b)（含边界条件）
`problem.solve(method)`	组装并求解
`problem.compute_error(u, exact,norm)`	单个误差范数
`problem.compute_errors(u, exact,grad)`	全部误差范数
`MixedPoissonProblem(...)`	返回 (Q,P)
`mixed.error_norms(P,Q,p_exact,q_exact)`	压力/通量误差
`mixed.check_conservation()`	单元质量守恒误差
`mixed.flux_at_points(points)`	给定点上的通量重构（自动定位单元）
`mixed.cell_flux(cell)`	单元形心处的通量向量（画箭头用）

方法	说明
`mixed.solve(method)`	鞍点系统的三种求解方式：`direct` / `schur` / `minres`

A.5 求解器与后处理

方法	说明
`Solver.direct(A,b)`	直接法
`Solver.conjugate_gradient(...)`	CG
`Solver.gmres(A,b,tol)`	GMRES
`Solver.bicgstab(A,b,tol)`	BiCGSTAB
`Solver.minres(A,b,tol)`	MINRES
`Solver.solve(A,b,method)`	统一入口
`Solver.apply_dirichlet(...)`	强加 Dirichlet
`Solver.recommend(n, symmetric)`	推荐求解器
`convergence_study(...)`	收敛性研究
`estimate_convergence_rate(h,e)`	收敛阶
`plot_solution(u,mesh,V,...)`	画解
`plot_mesh(mesh,...)`	画网格
`plot_error(u,mesh,space, exact)`	画误差
`plot_convergence(h,errors)`	画收敛曲线
`plot_pressure_2d(mesh,P)`	画 RT0 压力
`plot_flux_2d(space=V,Q=Q)`	画通量（用 q_h 在单元形心处重构）
`compare_solutions(entries)`	并排比较

附录 B　数学符号

B.1　一般记号

符号	含义
Ω	求解区域
$\partial\Omega$	区域边界
d	空间维数（1、2、3）
h	网格尺寸（最大单元直径）
K	网格单元
$\hat{K}$	参考单元
n	自由度个数
$\mathcal{D}$	Dirichlet自由度集合

B.2　函数空间

符号	含义
V	无穷维试探/检验空间
$V_h \subset V$	有限维离散空间（有限元或虚拟元）
$H^1(\Omega)$	一阶 Sobolev 空间
$H(\mathrm{div};\Omega)$	散度平方可积的向量场空间
$L^2(\Omega)$	平方可积函数空间
$\mathcal{P}_k$	次数不超过 k 的多项式空间
Q_k	每个方向次数不超过 k 的张量积多项式空间

B.3　算子与范数

符号	含义
∇u	梯度
$\nabla \cdot q$	散度
Δu	Laplace 算子
φ_i / $\hat{\varphi}_i$	物理 / 参考单元上的基函数
Π^{∇}	VEM 的梯度投影算子
Π_{dof}	投影算子在自由度上的矩阵表示
A_{proj}, S_{stab}	VEM 单元矩阵的一致性项与稳定化项
J_K	参考单元到物理单元的雅可比矩阵
$\|e\|_{L^2}$, $\|e\|_{H^1}$, $\|e\|_{L^\infty}$	误差范数
q, p	RT0 混合格式中的通量与压力
A, b	离散线性系统的矩阵与右端项

附录 C　参考文献

参考文献

[1] S. C. Brenner and L. R. Scott, *The Mathematical Theory of Finite Element Methods*, 3rd ed., Springer, 2008.

[2] A. Ern and J.-L. Guermond, *Theory and Practice of Finite Elements*, Springer, 2004.

[3] A. Logg, K.-A. Mardal, and G. Wells (eds.), *Automated Solution of Differential Equations by the Finite Element Method*, Springer, 2012.

[4] L. Beirão da Veiga, F. Brezzi, A. Cangiani, G. Manzini, L. D. Marini, and A. Russo, "Basic principles of virtual element methods", *Mathematical Models and Methods in Applied Sciences*, 23(1):199–214, 2013.

[5] L. Beirão da Veiga, F. Brezzi, L. D. Marini, and A. Russo, "The Hitchhiker's Guide to the Virtual Element Method", *Mathematical Models and Methods in Applied Sciences*, 24(8):1541–1573, 2014.

[6] L. Beirão da Veiga, F. Brezzi, L. D. Marini, and A. Russo, "Virtual element methods for general second order elliptic problems on polygonal meshes", *SIAM Journal on Numerical Analysis*, 2013.

[7] F. Brezzi and M. Fortin, *Mixed and Hybrid Finite Element Methods*, Springer, 1991.

[8] P.-A. Raviart and J.-M. Thomas, "A mixed finite element method for 2nd order elliptic problems", in *Mathematical Aspects of Finite Element Methods*, Springer, 1977.

[9] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, 2003.